

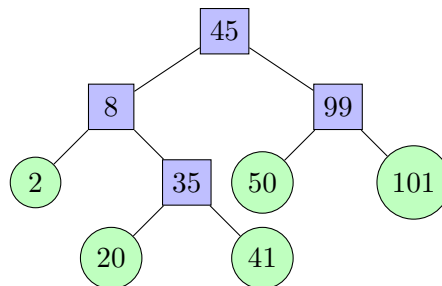
MPI* – TP : Arbres binaires de recherche et arbres 2-3

1 Arbres binaires de recherche

On rappelle que les *arbres binaires de recherche* (ABR) se définissent récursivement comme étant :

- soit une feuille
- soit un nœud ayant deux fils : un fils gauche et un fils droit qui sont aussi abr. De plus les étiquettes des feuilles du fils gauche doivent être inférieures ou égales à l'étiquette de la racine et les étiquettes des feuilles du fils droit strictement supérieures.

Ceci est un ABR.



On utilisera le type

```

type abr =
  | Feuille of int
  | Noeud of abr * int * abr
;;
  
```

1. Écrire une valeur Caml représentant l'abr dessiné ci-dessus.
2. Écrire une fonction `verif_abr : abr -> bool` vérifiant si un abr est un correct.
3. Écrire et tester les fonctions usuelles de manipulation d'un abr
 - (a) `existe : int -> abr -> bool`
 - (b) `insere : int -> abr -> abr`
 - (c) `supp : int -> abr -> abr`
 - (d) `minmax : abr -> (int * int)`
4. Écrire une fonction de tri de liste d'entiers reposant sur les abr.
5. Rappeler l'inconvénient principal lorsqu'on utilise la structure d'arbre binaire de recherche.

2 Arbres 2-3

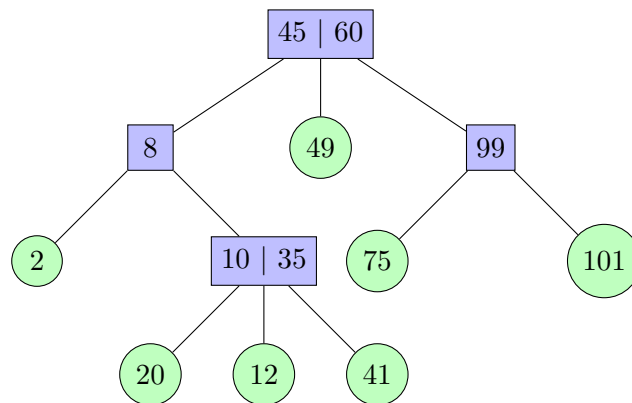
Les arbres 2-3 sont une amélioration des abr. Il existe maintenant 2 types de nœuds :

- les nœuds-2 ayant une étiquette n et deux fils g et d comme dans les abr ;
- les nœuds-3 ayant 2 étiquettes $n < m$ et trois fils g, m et d .

Les nœuds-2 vérifient les mêmes contraintes d'étiquettes que celles des arbres binaires. Pour les nœuds-3, soient x, y, z des étiquettes des fils g, m et d respectivement, alors elles doivent vérifier

$$x \leq n < y \leq m < z.$$

Voici un exemple d'arbre 2-3.



On utilisera le type

```
type arbre23 =
  | Feuille of int
  | Deux of arbre23 * int * arbre23
  | Trois of arbre23 * int * arbre23 * int * arbre23
;;
```

Arbres 2-3 équilibrés À partir de maintenant, on ajoute la contrainte qu'un arbre 2-3 possède ses n feuilles à la même profondeur p . Montrer la relation

$$\log_3 p \leq n \leq \log_2 p.$$

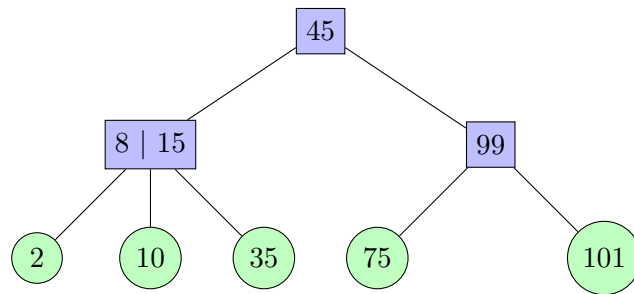
Écrire une fonction `verif : arbre23 -> true` vérifiant qu'un `arbre23` est bien un arbre 2-3 équilibré.

2.1 Parcours d'un arbre 2-3

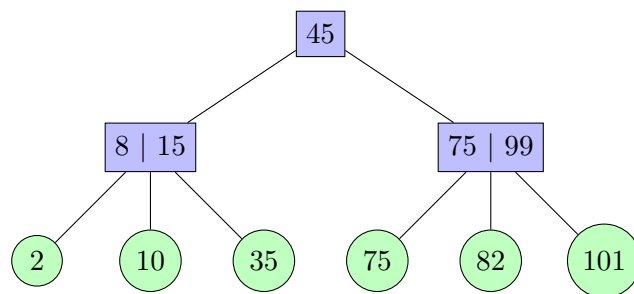
Écrire une fonction de parcours en profondeur des feuilles d'un arbre 2-3. Dans quel ordre apparaissent les feuilles ?

2.2 Insertion

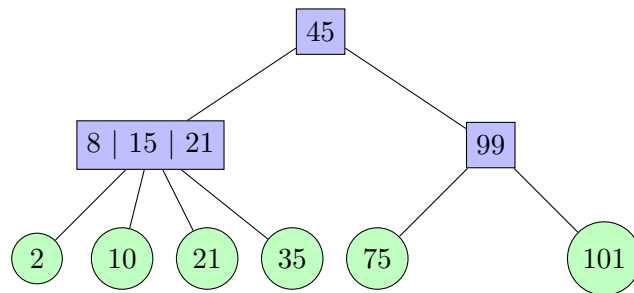
Pour insérer une nouvelle feuille dans un arbre 2-3 équilibré on procède comme pour les abr en descendant à l'endroit où devrait apparaître la feuille. Il y a alors deux possibilités : tomber sur un nœud-2 ou tomber sur un nœud-3. Si on tombe sur un nœud-2 il suffit de le transformer en nœud-3. Par exemple, ajouter 82 dans



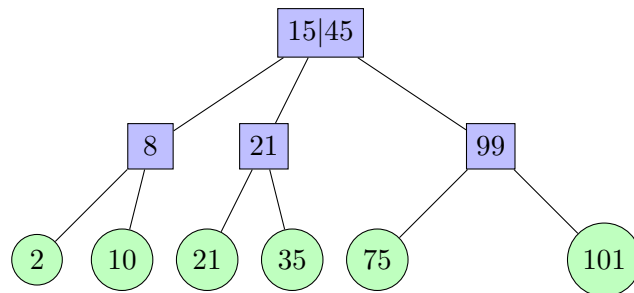
conduit à



Le cas plus difficile est donc celui où il faut ajouter une feuille à un nœud-3. Par exemple si on veut ajouter 21 au lieu de 82 dans l'exemple précédent.



On voit alors qu'on crée un nœud-4 ce qui n'est pas permis. L'idée est alors d'**éclater** ce nœud-4 en 2 nœuds-2. Si le père du nœud-4 était un nœud-2 pas de problème sinon il faut continuer à éclater jusqu'à ce que ça marche. Dans le pire cas, on arrive à la racine qu'il faudra éclater en 2 nœuds-2 : la profondeur de l'arbre augmente de 1 dans ce cas. Dans notre exemple il n'y a pas de soucis particulier puisque le père est un nœud 2. On obtient alors :



Pour gérer facilement le cas de l'éclatement on introduit une **exception** Caml :

```
exception Eclate of arbre23 * int * arbre23;;
```

Cette exception garde la trace des 2 arbres nœud-2 à remonter ainsi que la valeur qui les sépare (dans l'exemple précédent : 15).

En Caml on utilise les exceptions ainsi :

```
raise Eclate (g, x, d)
```

est utilisé pour déclencher l'exception.

```
try
  expr1
with
  Eclate (g, x, d) -> expr2
```

Tente d'évaluer **expr1** et si une exception **Eclate** est déclenchée alors elle est *rattrapée* c'est-à-dire que c'est l'expression **expr2** qui est évaluée.

Écrire et tester une fonction `insere : int -> arbre23 -> arbre23` insérant une feuille dans un arbre23 équilibré.

2.3 Tri arbre 2-3

Déduire de ce qui précède une fonction `tri_arbre23 : int list -> int list` triant une liste d'entiers. Quelle est sa complexité ?

2.4 Suppression (difficile)

La suppression dans un arbre 2-3 est une opération nettement plus complexe. Je laisse le soin aux élèves les plus dégourdis de s'attaquer à cette question.

Voici quelques pistes :

- Pour supprimer une feuille d'un nœud-3 : pas de problème particulier
- S'il s'agit d'un nœud-2 il faut considérer le grand-père de la feuille. Il y a deux cas A/ c'est un nœud 3, B/ c'est un nœud 2.
- Dans le cas A/ on regarde ses frères, si l'un deux est un nœud 2 on fusionne avec lui sinon on délocalise vers nous une des branches voisines.
- Dans le cas B/ on regarde aussi son frère, s'il est un nœud 3 on délocalise vers nous une de ses branches.
- Sinon on est dans le cas le plus difficile : on fusionne avec notre frère qui est un nœud 2 et on devient un nœud-3. Mais le grand-père n'a plus qu'un seul fils ! Il faut alors remonter la procédure de suppression plus haut dans l'arbre. Ceci peut-être fait à l'aide d'une exception `Fils_unique of arbre23`.
- Dans le pire cas on remonte un fils unique jusque la racine qu'il suffit de supprimer : la profondeur de l'arbre 2-3 diminue d'une unité.